SFAD

Screen First AI Development

画面を起点に、AIと協働して、業務システムを構築する開発メソドロジ
ー

Zero Time
合意の即時化

Zero Effort
作業の自動化

Zero Mistakes
ミスの排除

Harmonic Insight

こんな経験、ありませんか？

仕様書を100ページ書いたのに
→「想定と違う」と言われた

アジャイルを導入したのに
→ 儀式だけが増えて速くならなかった

Excelで回している業務を
→ システム化したいが予算も時間もない

SFADは、この3つの問題を構造的に解決します。

REALITY CHECK

メソドロジーという幻想

海外から輸入された開発手法は、なぜ日本で機能していないのか

アジャイルの「約束」

- ~~スプリントで小さく速く回す~~
- ~~ペアプログラミングで品質を担保~~
- ~~デイリースタンドアップで進捗を共有~~
- ~~レトロスペクティブで継続的に改善~~
- ~~ユーザーストーリーで顧客の声を反映~~

→ 結果: 単なる「短期プロトタイプ手法」に矮小化。儀式だけが残った。

現場の「現実」

プロジェクトごとにやり方が違う	再現性ゼロ
個人が勝手にプロセスを変える	標準化ゼロ
ドキュメントが毎回バラバラ	品質保証ゼロ
「アジャイルです」と言うだけ	実態は場当たり開発
メソドロジーを借りただけ	本来の導入とは程遠い

THE GAP

アジャイルは進め方を変えた。
でも作り方は変えていない。
だから現場は変わらなかった。

進め方

（プロセス管理）

アジャイル
スクラム
カンバン

作り方

（構築手法）

← 空白だった

SFADは、この「作り方」を埋めるメソドロジーです。

THE REAL QUESTION

本来メソドロジーに求められるもの

「誰がやっても、同じプロセスで、同じ品質を、再現できる」
それがメソドロジーの本来の役割。

再現性
Reproducibility

担当者が変わっても、プロジェクトが変わっても、
同じ手順で同じ品質のアウトプットが出る。

標準プロセス
Standardized Process

10ステップの固定フローで、
「次に何をすべきか」に迷わない。

統一ドキュメント
Unified Documents

画面モック・要件一覧・権限マトリクスなど、
成果物のフォーマットが標準化されている。

品質保証
Quality Assurance

各ステップの完了条件が明確。
「何ができたら次に進めるか」が定義されている。

SFADは、この4つの要件をすべて満たすために設計された。

WHAT

SFADとは何か

画面を最初に作り、AIと協働しながら、業務システムを段階的に完成させる開発手法。

顧客が「実際の画面」で合意形成を行い、AIがコード生成・修正・テストを支援し、開発のスピード・品質・コストを同時に最適化する。

Zero Time

合意形成の即時化

画面が仕様書そのもの。
テキストで説明する時間を排除。
顧客は見たままに合意できる。

Zero Effort

作業の自動化

AIがコード生成・Excel取込・
軽微な修正を自動化。
開発者は判断と設計に集中。

Zero Mistakes

認識ズレの排除

画面ベースの合意で「想定と違う」を
防止。AIによる自動テスト・修正で
人的ミスを排除。

PRINCIPLES

5つの核心原則

#1	**Screen First** 画面が仕様書	要件定義の前に画面モックを作成。顧客と「見える形」で合意する。
#2	**Mock First** モックで先に動かす	DB/APIの前にモックデータで画面を完成。実データ投入前に完成形を確認。
#3	**Excel Boundary** Excelが架け橋	既存Excelをそのまま取込・表示。現行業務を壊さずにシステム化。
#4	**AI Co-pilot** AIが開発パートナー	構成提案・コード生成・修正をAIが担当。開発者は判断とレビューに集中。
#5	**Central Backlog** 全体像をトップに集約	要件・進捗・改善要望をトップページに集約。顧客と全体像を常に共有。

COMPARISON

アジャイル × SFAD = 共存できる

	ウォーターフォール	アジャイル	ローコード	SFAD
顧客が画面を見る時	実装完了後	スプリント末	設定時	最初 (Step 1)
合意の手段	テキスト仕様書	ユーザーストーリー	設定画面	実画面モック
AI活用	なし	部分的	なし	全工程で協働
Excel連携	別途開発	別途開発	コネクタ依存	標準 I/O
再現性	手順書依存	チーム依存	テンプレート	10Step固定
手戻りコスト	極めて高い	中程度	低い	最小
要件変更対応	困難	対応可能	設定変更	即時 (AI修正)
適用対象	大規模SI	Web / アプリ	定型業務	業務システム全般

アジャイルは「進め方」。SFADは「作り方」。敵ではなく、補完する関係です。

PROCESS

10ステップ — プロセス全体像

1	2	3	4	5	6	7	8	9	10
レイアウト カラー選択	TOP Page Backlog	メニュー 構成決定	画面構成案 作成	Excelデータ アップロード	優先度順に 実装	DB移行 リファクタ	改善要望 AI修正	ユーザー アクセス権	ログイン 権限組込
			AI	AI	AI		AI		
SETUP			BUILD CYCLE			MIGRATE	OPERATE	LAUNCH	

Step 4 → 5 → 6 を機能ごとに繰り返す
全機能完了後に Step 7 へ進む

Step 8 改善要望 → Step 6 要件にフィードバック
リリース後の運用改善ループ

AI CO-PILOT

全ステップにおけるAIの関与

AIが提案し、人間が承認する。だから誰がやっても同じ結果になる

1	**初期設定**	業種・規模に応じたテンプレート・レイアウトの推薦	推薦
2	**Backlog構築**	過去パターンからKPI・要件テンプレートを自動生成	生成
3	**メニュー構成**	ヒアリング内容から機能グループ案を自動分類・提案	提案
4	**画面構成案**	各機能の画面構成（一覧/入力/ダッシュボード等）を自動起案	起案
5	**Excel取込**	カラムマッピングの自動推定。データ型の判定・変換	自動化
6	**機能実装**	画面構成案 → 実装コードの生成。テスト自動生成	実装
7	**DB移行**	ExcelスキーマからDBスキーマ（Prisma）を自動設計	設計
8	**改善要望**	軽微な修正はAIが直接コード修正 → レビュー → デプロイ	修正
9	**アクセス権**	ロール・権限マトリクスからガード実装コードを生成	生成
10	**本番化**	全画面の権限チェック。操作ログ設計。セキュリティ検証	検証

WHY REPRODUCIBLE

なぜ「誰がやっても同じ」が実現できるのか

AIが提案し、人間は「選ぶ」「承認する」だけ。判断をAIに委ねることで属人性を排除。

AIがやること
推薦する
起案する
推定する
生成する
設計する
修正する

→

人間がやること
選ぶ
承認する
確認する
レビューする

≠

従来の「作り方」
設計する
考える
書く
個人の力量次第

この設計はAI以前には不可能だった。SFADが今のタイミングでしか成立しない理由。

TECH STACK

SFADを支える技術基盤

メソドロジーの再現性を担保する、標準化された技術スタック

AI開発 **Claude Code**	AIによるコード生成・修正・レビューの中核エンジン。 全10ステップを通じて開発者と協働する。	AI Partner
フレームワーク **Next.js**	React ベースのフルスタックフレームワーク。 SSR/SSG対応、API Routes、App Router。	Framework
ホスティング **Vercel**	Next.js のネイティブホスティング。 プレビューデプロイで顧客確認が即座に可能。	Deploy
ソース管理 **GitHub**	コード管理・PR レビュー・Issue トラッキング。 Claude Code との統合でAI修正のワークフローを実現。	Source
データベース **Neon**	サーバーレス PostgreSQL。Step 7 のDB移行先。 Prisma ORM で型安全なデータアクセス。	Database

SCOPE

対象と適用領域

BEST FIT

業務システムの新規構築
社内ポータル、資産管理、勤怠、経費精算などの業務アプリ

Excel業務のシステム化
Excelで運用している業務を段階的にWebアプリへ移行

中小規模のSI案件
5〜50画面規模。顧客と密なコミュニケーションが可能

プロトタイプ → 本番化
PoCからそのまま本番システムに育てる開発スタイル

NOT IDEAL

リアルタイムシステム
ミリ秒単位のレスポンスが求められる取引所・制御系

大規模基幹システム
1000画面超のERP。初期から全体設計が必要な案件

UI/UXが差別化要因
BtoCアプリなど独自のUI体験が競争優位になる製品

TARGET

SIer / 受託開発ベンダ
合意形成スピードを向上

社内IT / 情シス
Excel業務のシステム化

AI活用を推進する組織
プロセスへのAI標準化

DEMO 1/2

セットアップ — 開発初日の画面

SFADでは、開発初日からこの画面が動きます。

Step 1: レイアウト・カラー選択

Step 2: Central Backlog（トップページ）

DEMO 2/2

ビルドサイクル — 画面構成と要件管理

画面構成案を作り、優先度順に実装を進めます。

Step 4: 各機能の画面構成案

Step 6: 要件を優先度順に管理

WHY SFAD

従来開発の課題と、SFADのアプローチ

業務システム構築で繰り返されてきた問題を、SFADはどう変えるか

Before（従来開発）

- 顧客が画面を見るのは実装後
- 仕様書ベースで認識ズレが多発
- 手戻りが常態化
- Excel連携は毎回別途開発
- 担当者が変わるとやり方も変わる
- プロセスが属人的で再現性がない

After（SFAD）

- 初日から画面を見せて合意
- 画面モックが仕様書を兼ねる
- 認識ズレが構造的に起きない
- Excel取込が標準機能
- 10ステップ固定で誰でも同じ進め方
- AIが生成・修正・テストを担う

「やり方を変える」のではなく「やり方が決まっている」。それがSFADです。

SFAD

Screen First AI Development

アジャイルは「進め方」の話。SFADは「作り方」の話。

画面から作る。AIと育てる。Excelで繋ぐ。

誰がやっても同じ10ステップで、同じ品質を再現できる。

誰がやっても同じ10ステップで、同じ品質を再現できる。

Zero Time | **Zero Effort** | **Zero Mistakes**

Harmonic Insight

PACKAGE

SFADスターターパッケージ

メソドロジーだけではなく、すぐに始められるツール一式をご提供します

1. **実践ガイドブック（PDF）**
   10ステップの詳細手順・判断基準・完了条件を網羅した5章構成のガイド
2. **スターターキット（GitHub）**
   テンプレート＋Excel I/Oライブラリ＋コンポーネント＋CLAUDE.md
3. **インフラ構築マニュアル（PDF）**
   GitHub・Vercel・Claude Code・Neonのセットアップ手順書
4. **テンプレート集（Excel 5種）**
   要件一覧・画面構成案・権限マトリクス・改善要望・プロジェクト計画
5. **動画教材（10本）**
   各ステップのデモ＋Claude Codeの実践的な使い方

導入コンサルティングの一環としてご提供

まずは、画面を見てください。

SFADの「Screen First」を、あなた自身の目で確かめてください。

無料デモを申し込む

実際のスターターキットを動かしながら
ご説明します

資料をダウンロードする

SFADの詳細資料をPDFでお送りします

お問い合わせ

導入相談・見積もりのご依頼はこちら

insight-office.com

Harmonic Insight